DRAFT - not verified by the authors - do not circulate

AIPIR: An AI-Enabled Predictive Infrastructure Resilience Framework for Energy-Sector Control Systems, with an Open Validation on a Power-Generation Testbed

Friday Ogochukwu Ikwuogu[1]*, Abidemi Orimogunje[2], Eria Othieno Pinyi[3], David Mike-Ewewie[4]

1 Independent Researcher, Odessa, Texas, USA; Friday.ikwuogu@gmail.com; ORCID 0009-0009-2222-1318
2 Electrical and Electronic Engineering Department, Redeemer's University, Ede, Osun State, Nigeria; orimogunjea@run.edu.ng
3 Computer Science and Engineering Department, University of Fairfax, USA; eriaothienopinyi@gmail.com
4 Computer Science Department, University of Texas Permian Basin, Odessa, Texas, USA; mike_d63291@utpb.edu
** Corresponding author: Friday Ogochukwu Ikwuogu, Friday.ikwuogu@gmail.com*

Abstract

Energy-sector control systems are monitored mostly by fixed alarm limits, which catch gross excursions but react late to subtle manipulation and say little about where to respond. We present AIPIR, an open, cloud-native framework that links four layers: Observe (telemetry exposed as Prometheus metrics), Predict (a ridge forecaster that predicts every sensor one second ahead and scores robustly normalized forecast residuals), Respond (thresholds calibrated on normal data only, debounced alarms routed to the affected sub-process), and Recover (Alertmanager-triggered playbooks and Terraform/Kubernetes definitions that restore the monitoring stack to a known-good state). We validate the detection layers on the open HAI security datasets, a hardware-in-the-loop testbed emulating steam-turbine power generation and pumped-storage hydropower, against static limits, PCA residual and Isolation Forest baselines, with every threshold set to the same false-alarm budget of one alarm episode per day on held-out normal data and all design choices frozen on a single development file. On 45 held-out attacks in HAI 21.03 (4.2 days), AIPIR detected 35 (78%) with a median time-to-detect of 16 s, against 19 s for static limits and 33 s for PCA; it achieved the highest point-wise F1 (0.69 vs 0.68 and 0.57), 0.72 false alarm episodes per day, and routed 100% of detected attacks to an attacked process. It did not detect the most attacks: PCA and static limits detected 40 and 37. On the older HAI 20.07 release all detectors were weaker (AIPIR 21/38 events). In a downtime scenario with assumed recovery times, AIPIR detection plus automated recovery reduced total downtime by 19.4% (range 0.6-49.9% across assumptions) relative to static alarms with manual recovery, whereas AIPIR detection with manual recovery increased it by 2.5% because it missed more attacks; an exploratory layered configuration running both detectors reduced it by 49.5%. Code, infrastructure definitions, models and the replication package are openly released.

Keywords: industrial control systems; anomaly detection; critical infrastructure resilience; energy sector; predictive analytics; observability; infrastructure as code

1 Introduction

Generation plants, pipelines and substations depend on industrial control systems (ICS) whose availability is a public-safety concern. Attacks on such systems are no longer hypothetical: the 2015 attack on Ukrainian distribution utilities combined intrusion with manipulation of operator workstations to cause outages for customers [7]. Guidance for operational technology (OT) security now emphasizes continuous monitoring, anomaly detection and practiced recovery alongside prevention [6].

In many plants, the monitoring that operators actually act on is still the alarm system: fixed high and low limits per point. Limits are simple, explainable and cheap, but an attacker who keeps each variable inside its band while breaking the relationships between variables can stay below them, and a limit alarm says which

point is out of range rather than which process to act on. Research on learned anomaly detection for ICS has advanced quickly with open testbed datasets such as SWaT [8] and HAI [1, 2], and with detectors based on reconstruction, prediction or deep sequence models [3, 4, 9]. Fewer studies connect detection to what happens next: how an alert reaches the right responder, and how the monitoring infrastructure itself is recovered, which is where downtime is lost or saved.

This paper contributes (i) AIPIR, an open framework that wires a predictive detector into standard observability and Infrastructure-as-Code (IaC) tooling, so that detection, alert routing and recovery are one reproducible system; (ii) a validation of its detector on two releases of the HAI power-generation testbed under a protocol that fixes the false-alarm budget for every detector and freezes design choices before held-out scoring; (iii) an explicit, assumption-swept scenario model that translates measured detection performance into downtime; and (iv) an honest account of where the predictive approach does not outperform simpler baselines.

2 Related work

ICS anomaly detection is commonly evaluated on testbed datasets with labelled attacks. SWaT [8] and HAI [1, 2] are the most used; HAI is distinctive for coupling real boiler, turbine and water-treatment controllers through a simulated steam-turbine and pumped-storage hydropower plant, which makes it a natural proxy for the energy sector. Detection methods range from residuals of linear projections (PCA) and isolation-based outlier scores [5] to forecasting with recurrent or convolutional networks [3, 9] and adversarially trained autoencoders [4]. Forecast-residual methods are attractive because the residual of each sensor is itself an attribution signal. Our detector deliberately uses a linear forecaster so that it trains in seconds on a CPU, can be audited, and runs as a small service.

On the operations side, site reliability engineering practice treats monitoring and alerting as code and measures recovery explicitly [10], and IaC makes infrastructure reproducible from declarations [11]. The MITRE ATT&CK for ICS knowledge base catalogues adversary techniques that motivate monitoring both process values and the monitoring path itself [12]. AIPIR applies these practices to the detection stack of an OT environment.

3 Methods

3.1 Framework

AIPIR has four layers (Fig. 1). Observe reads plant telemetry once per second; in this study it replays HAI files, and in a deployment it would read an OPC-UA server or historian. Each second the service publishes the anomaly score, the threshold, a per-process alarm flag and per-sensor residuals as Prometheus metrics. Predict computes the score with the streaming form of the detector described below. Respond is a Prometheus alert rule that fires once per routed process; Alertmanager forwards it to the operator channel and to the service's recovery webhook. Recover selects the playbook for the routed process (boiler, turbine or water treatment) and, in this release, records it in dry-run mode only; separately, the whole detection stack is declared in Terraform and Kubernetes manifests with two replicas, health probes, a disruption budget, revision history for rollback, and a network policy that gives the detector no egress, so a failed or suspect detector is restored by re-applying the declarations.

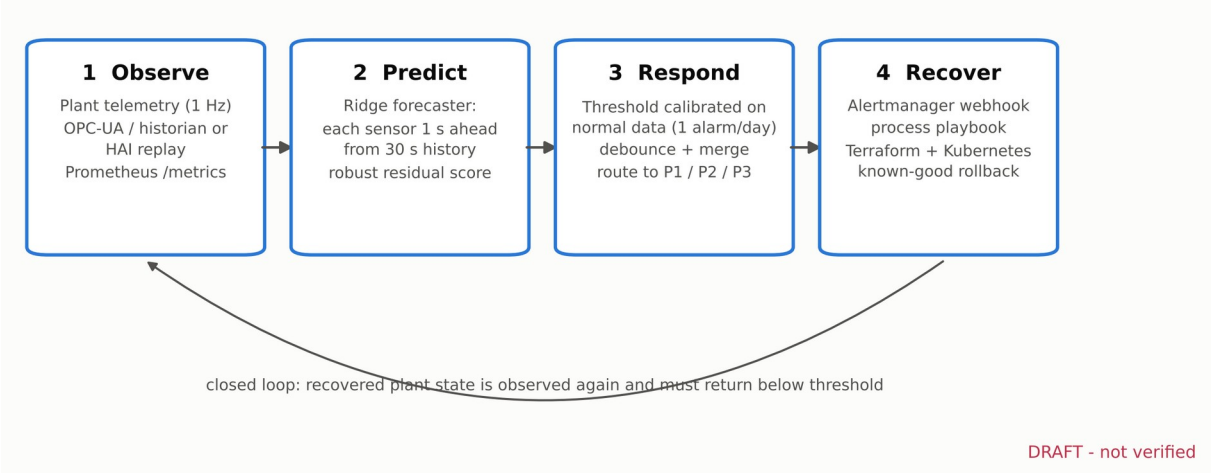


Fig. 1. AIPIR layers. All components are open source and declared as code (deploy/).

3.2 Data

We use HAI 21.03 as the primary dataset and HAI 20.07 for replication, both from the HAI repository at a pinned commit, licensed CC BY-SA 4.0 [1, 2]. Data are sampled at 1 Hz. HAI 21.03 provides three normal training files (921603 usable seconds) and five test files with 50 labelled attacks; HAI 20.07 provides two training and two test files with 38 attacks. The number of contiguous attack-label runs in every test file equals the attack count published by the dataset authors. Points that are constant in normal data were dropped (57 of 79 points used for HAI 21.03, 51 of 59 for HAI 20.07). One HAI 20.07 training file contains 776 attack-labelled seconds; these, plus a 10-minute recovery margin, were excluded from training.

3.3 Detectors

D0 Static limits. Per-point minimum and maximum of normal data; the score is the largest exceedance, in units of the point's normal range. **D1 PCA residual.** Squared reconstruction error of a PCA model keeping 95% of normal variance, smoothed over 60 s. **D2 Isolation Forest** [5] with 100 trees on a 60,000-row sample, smoothed over 10 s. **D3 AIPIR predictive.** A multi-output ridge regression predicts every standardized point at second t from all points at lags of 1, 2, 3, 5, 10, 20 and 30 s. Each point's absolute forecast error is divided by its 99.9th percentile on normal data and clipped at 5, so that a legitimate setpoint step on one point cannot dominate; residuals are averaged over a trailing 10 s window and the score is the mean of the five largest. All smoothing is causal, so scores use only past data.

Two ablations test the design: adding a "consistency" channel that predicts each point from the current values of all others (D3+C), and removing robust normalization (95th-percentile scaling, no clipping; D3-noclip). Models were fitted with scikit-learn [13].

3.4 Calibration, alarms and evaluation protocol

For every detector the first 80% of each normal segment fits the model and the last 20% (51.2 h for HAI 21.03) calibrates the threshold: the lowest threshold that produces at most one alarm episode per day on that normal data [VERIFY V2]. An alarm episode is a run of above-threshold seconds, with runs closer than 30 s merged and episodes shorter than 5 s discarded (debounce) [VERIFY V3]. Test labels are never used for thresholds.

Design choices for all detectors were made on HAI 21.03 test1 (five attacks) and frozen before HAI 21.03 test2-test5 and both HAI 20.07 test files were scored. An attack event is detected if an alarm episode is confirmed between the attack's start and 60 s after its end; an alarm already active before the attack began

earns no credit. We report event recall, time-to-detect (TTD: confirmation time minus labelled start), point-wise precision, recall and F1, false alarm episodes per day (episodes overlapping no attack or its grace window), and routing accuracy (the alarm is routed to the process of its top-contributing P1-P3 point, and is correct if that process carries an attack label during the event).

3.5 Downtime scenario

To translate detection into downtime we use a deliberately simple scenario model, not a measurement. For each held-out attack, downtime equals TTD plus recovery time if the operating model detects it, or the time until a missed attack is discovered by other means plus manual recovery. Central assumptions are 4 h for manual recovery, 15 min for playbook/IaC-assisted recovery and 24 h to discover a missed attack, each swept over a range (manual 2-8 h, automated 5-30 min, discovery 8-72 h) [VERIFY V1]. We compare S0 static limits with manual recovery, S1 AIPIR with manual recovery, S2 AIPIR with automated recovery, and S3, an exploratory layered model in which static limits and AIPIR both run (detection at the earlier of the two; automated recovery when AIPIR detected). S3 was defined after the held-out results showed the two detectors miss different attacks and is reported as exploratory.

4 Results

Table 1 summarizes held-out results. On HAI 21.03, AIPIR detected 35 of 45 attacks (78%), fewer than PCA (40) and fewer than static limits (37). It detected them earliest: median TTD 16 s versus 19 s and 33 s (Fig. 3). Its point-wise F1 was 0.69 (PCA 0.68, static 0.57), and 0.72 false alarm episodes occurred per test day. Every detected attack with a process label was routed to an attacked process (n = 35), compared with 80% for PCA. Isolation Forest detected 0 of the HAI 21.03 attacks at this false-alarm budget.

Table 1. Held-out results (HAI 21.03 test2-5; HAI 20.07 test1-2). Thresholds calibrated to <= 1 alarm episode/day on normal data.

Dataset / detector	Detected	Event recall	Median TTD (s)	Precision	Recall	F1	False alarms/day	Routing acc.
HAI-21.03 - static limits	37/45	0.82	19	0.86	0.43	0.57	0.00	1.00
HAI-21.03 - PCA residual	40/45	0.89	33	0.72	0.64	0.68	0.48	0.80
HAI-21.03 - Isolation Forest	0/45	0.00	-	0.00	0.00	0.00	0.00	-
HAI-21.03 - AIPIR predictive	35/45	0.78	16	0.84	0.59	0.69	0.72	1.00
HAI-20.07 - static limits	20/38	0.53	70	0.07	0.69	0.13	0.00	1.00
HAI-20.07 - PCA residual	27/38	0.71	58	0.47	0.68	0.55	18.46	1.00
HAI-20.07 - Isolation Forest	25/38	0.66	120	0.53	0.22	0.32	16.13	-
HAI-20.07 - AIPIR predictive	21/38	0.55	68	0.99	0.29	0.45	0.39	0.86

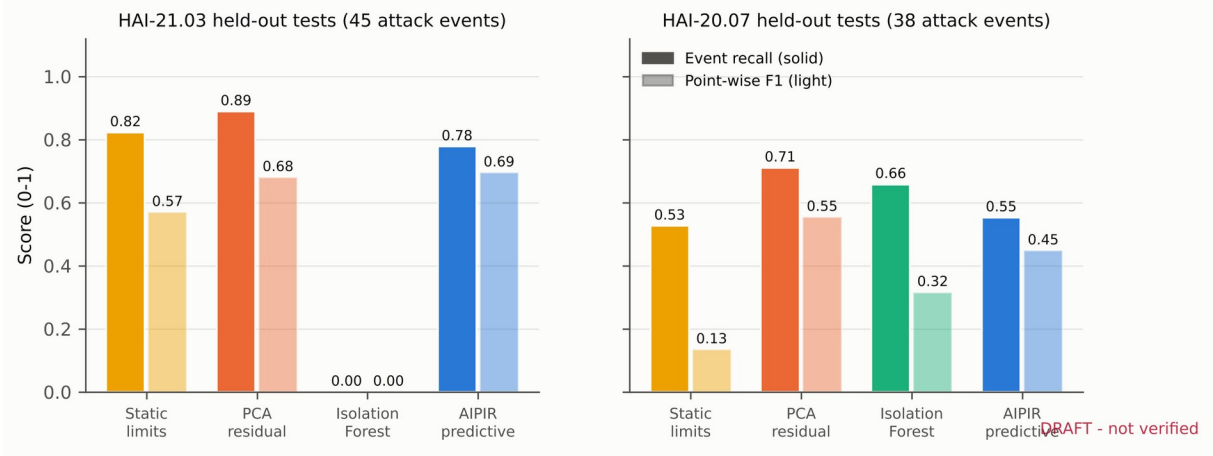


Fig. 2. Event recall (solid) and point-wise F1 (light) on held-out tests.

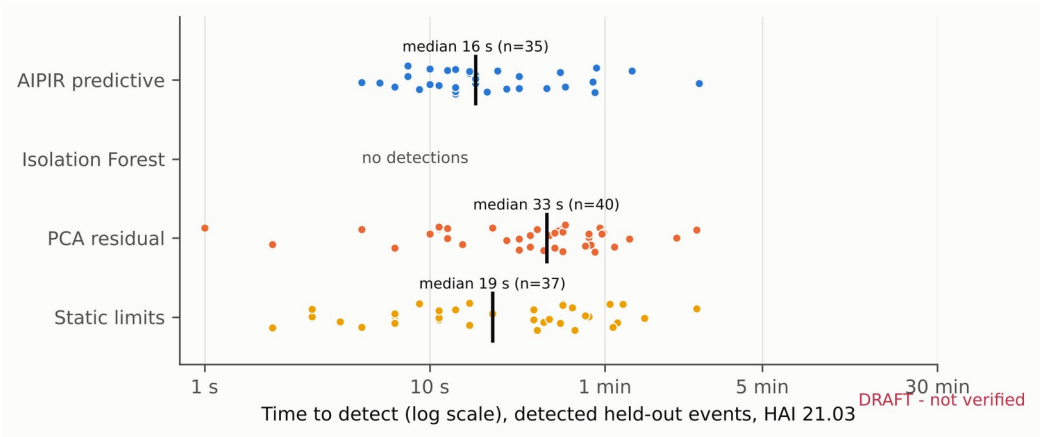


Fig. 3. Time to detect for detected held-out attacks, HAI 21.03. Black bars: medians.

On HAI 20.07 every detector was weaker, and calibration on normal data transferred poorly to the test files. AIPIR detected 21 of 38 attacks with precision 0.99 and 0.39 false alarm episodes per day. PCA detected 27 and Isolation Forest 25, but at 18.5 and 16.1 false alarm episodes per day, far above the one-per-day budget they met on normal data; static limits detected 20 with 0.00 false alarms per day. Fig. 4 shows that rankings also depend on the budget: at four alarms per day on normal data, AIPIR's event recall rises to 0.87 on HAI 21.03 and 0.68 on HAI 20.07, with realized test false-alarm rates of 4.6 and 8.0 per day. Reported comparisons should therefore always state the false-alarm budget and the realized rate together.

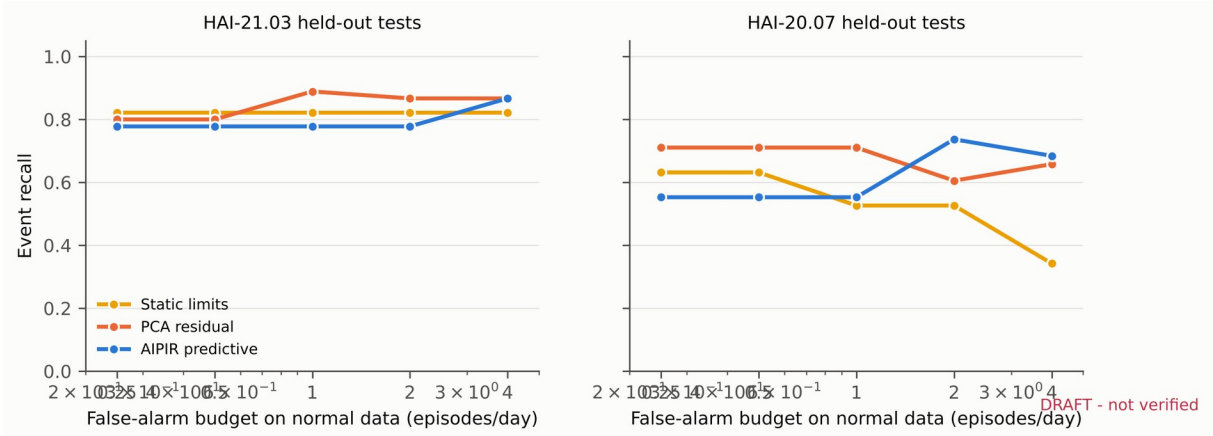


Fig. 4. Event recall as each detector's threshold is recalibrated to different false-alarm budgets (normal data only).

The ablations support the design choices made on the development file (Table 2). Without robust normalization, a few points with exogenous setpoint changes dominate the normal residual distribution, the calibrated threshold rises, and AIPIR detected only 12 held-out attacks. Adding the consistency channel left event detection at 35 but lowered F1 to 0.60, so it is not part of the default configuration. Fig. 5 shows the score on a six-hour excerpt.

Table 2. Ablations of the AIPIR detector.

Variant (HAI 21.03 held-out)	Detected	Median TTD (s)	F1	False alarms/day
AIPIR predictive	35/45	16	0.69	0.72
AIPIR + consistency channel	35/45	21	0.60	0.72
AIPIR without robust normalization	12/45	34	0.08	0.00

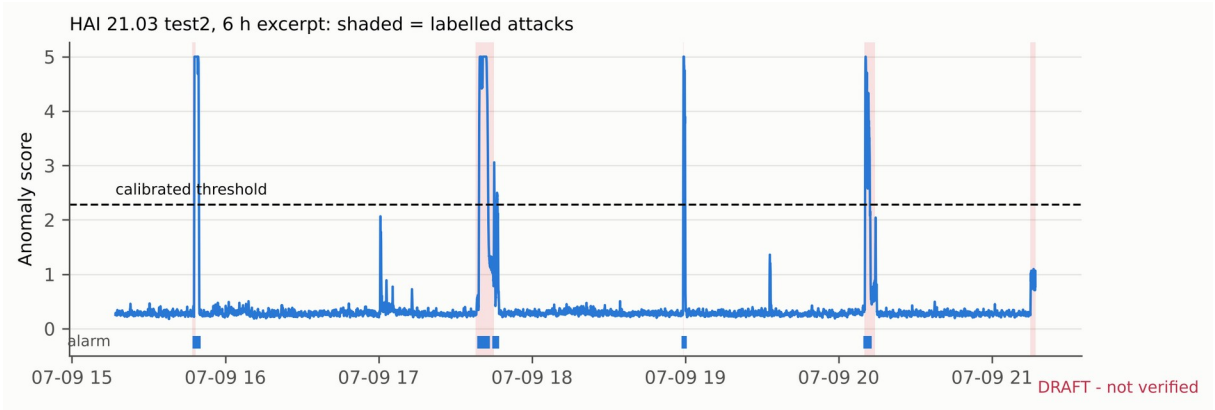


Fig. 5. AIPIR anomaly score, calibrated threshold and alarms over six hours of HAI 21.03 test2.

Detection and the downtime scenario. Static limits and AIPIR miss different attacks: on HAI 21.03, 4 attacks were detected only by static limits and 2 only by AIPIR; on HAI 20.07 the split was 8 and 9. Across all 83 held-out attacks, the scenario model (Fig. 6, Table 3) gives 957 h of total downtime for S0, 981 h for S1 (2.5% more), 771 h for S2 (19.4% less; 0.6-49.9% across all assumption combinations) and 483 h for the exploratory S3 (49.5% less; 43.0-60.9%). Most of the S2 gain comes from faster recovery once an attack is detected. Detection speed alone (S1) does not help: time-to-detect is measured in seconds while recovery and discovery are measured in hours, so the attacks AIPIR misses outweigh the seconds it saves. The lower end of the S2 range (0.6%) occurs when automated recovery is slow and manual recovery fast; the gain is therefore conditional on recovery actually being automated and rehearsed.

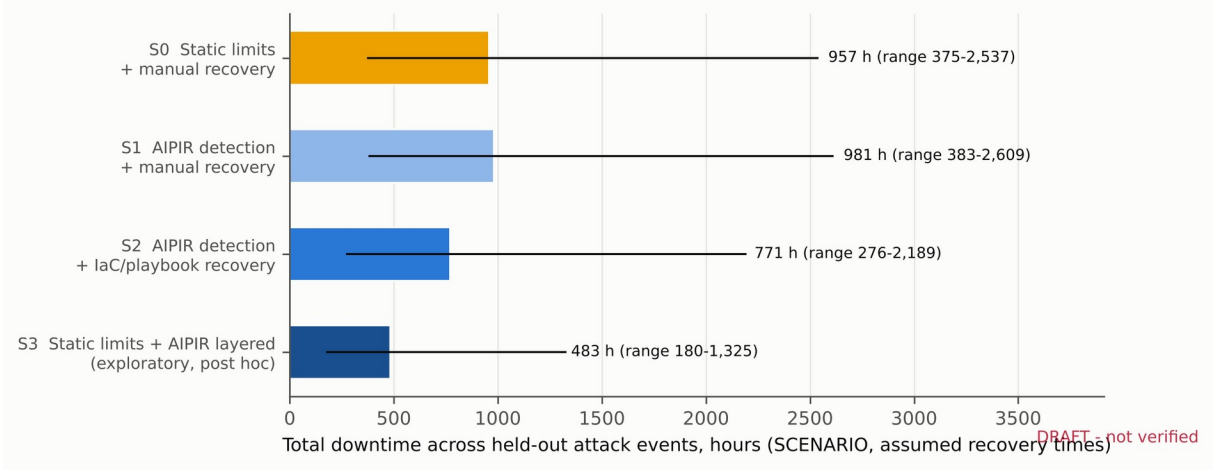


Fig. 6. Downtime scenario totals at central assumptions; lines show the range across all assumption combinations. Scenario model, not measurement.

Table 3. Downtime scenario over 83 held-out attacks (assumed recovery and discovery times; see 3.5).

Operating model	Total downtime (h), central	Change vs S0
S0 static limits + manual recovery	957	-
S1 AIPIR + manual recovery	981	2.5%
S2 AIPIR + automated recovery	771	-19.4%
S3 static limits + AIPIR, layered (exploratory)	483	-49.5%

Engineering checks. The streaming scorer used by the service reproduced the batch scores to a maximum relative difference of 1.7e-4 over one hour of telemetry. Batch scoring with AIPIR ran at about 196,081 telemetry-seconds per second on a 2-vCPU machine, and fitting took 26 s, so the detector is light enough for an edge node or a small cloud container.

5 Discussion

The results support a narrower claim than "AI beats alarms". On the HAI 21.03 power-generation testbed, a simple, auditable predictive detector detected attacks earlier than static limits and PCA, with the best balance of point-wise precision and recall and correct process routing, at a false-alarm rate within the budget (0.72 per day, against 0.00 for static limits and 0.48 for PCA); but it detected fewer attacks than either. On HAI 20.07 no detector was strong, and the detectors that found more attacks did so at many times the false-alarm budget. The practical reading is that a predictive layer complements, rather than replaces, limit alarms: they miss different attacks, and routing and speed are what an operator needs from the alert that arrives first.

The scenario model makes a second point visible: for downtime, what happens after detection dominates what happens before it. A detector that is seconds faster changes little if recovery takes hours; recovery that is scripted, rehearsed and declared as code changes a great deal, and missed attacks dominate the total. This argues for investing in the Respond and Recover layers, and for measuring recovery as carefully as detection. The recovery times here are assumptions; measured recovery drills would replace them.

Limitations are listed in full in the repository (docs/LIMITATIONS.md). The main ones are: a testbed rather than an operating plant; design choices made on one development file with five attacks; an exploratory layered model defined after seeing held-out results; recovery validated for configuration and data flow (dry-

run) rather than for effect on a process; and coarse interval labels. Future work will evaluate on newer HAI releases and other open testbeds, pre-register the layered model, add the eTaPR metric for comparability with competition results, and measure recovery in lab drills.

6 Conclusion

AIPIR shows how a lightweight predictive detector, standard observability tooling and Infrastructure-as-Code can be assembled into one reproducible resilience loop for energy-sector control systems, and it reports, under a fixed false-alarm budget and a frozen design, where that loop helps and where it does not. Everything needed to reproduce or extend the results, including the infrastructure definitions, is openly available.

Declarations

Data availability. HAI datasets are available at https://github.com/icsdataset/hai under CC BY-SA 4.0. Derived scores, events, models and figures are released with the code under CC BY-SA 4.0.

Code availability. All code, deployment definitions and the replication package: [repository URL] (MIT license), archived at [DOI].

Author contributions (CRediT). Friday Ogochukwu Ikwuogu: Conceptualization, Methodology, Software, Formal analysis, Validation, Writing - original draft. Abidemi Orimogunje: Validation, Writing - review and editing. Eria Othieno Pinyi: Validation, Writing - review and editing. David Mike-Ewewie: Validation, Writing - review and editing. [VERIFY V4]

Competing interests. The authors declare no competing interests. [VERIFY V4/V6]

Funding. This work received no specific funding. [VERIFY V4]

AI assistance. An AI assistant (Claude, Anthropic) was used to help write analysis code, deployment files and draft text. The authors designed the study, verified every result against the data and the pipeline outputs, made all analytic decisions, and take full responsibility for the content.

Use of data. This study used only public data. No employer, operator or customer data was used. [VERIFY V6]

References

[1] Shin, H.-K., Lee, W., Yun, J.-H., Min, B.-G. Two ICS security datasets and anomaly detection contest on the HIL-based augmented ICS testbed. Cyber Security Experimentation and Test Workshop (CSET '21), ACM, 2021, 36-40. doi:10.1145/3474718.3474719

[2] Shin, H.-K., Lee, W., Yun, J.-H., Kim, H. HAI 1.0: HIL-based augmented ICS security dataset. 13th USENIX Workshop on Cyber Security Experimentation and Test (CSET '20), 2020.

[3] Hundman, K., Constantinou, V., Laporte, C., Colwell, I., Soderstrom, T. Detecting spacecraft anomalies using LSTMs and nonparametric dynamic thresholding. KDD 2018, 387-395. doi:10.1145/3219819.3219845

[4] Audibert, J., Michiardi, P., Guyard, F., Marti, S., Zuluaga, M. A. USAD: UnSupervised anomaly detection on multivariate time series. KDD 2020, 3395-3404. doi:10.1145/3394486.3403392

[5] Liu, F. T., Ting, K. M., Zhou, Z.-H. Isolation forest. IEEE International Conference on Data Mining (ICDM), 2008, 413-422. doi:10.1109/ICDM.2008.17

[6] Stouffer, K., Pease, M., Tang, C., Zimmerman, T., Pillitteri, V., Lightman, S., Hahn, A., Saravia, S., Sherule, A., Thompson, M. Guide to Operational Technology (OT) Security. NIST SP 800-82 Rev. 3, 2023. doi:10.6028/NIST.SP.800-82r3

[7] Lee, R. M., Assante, M. J., Conway, T. Analysis of the cyber attack on the Ukrainian power grid. SANS ICS and E-ISAC, 2016.

[8] Goh, J., Adepu, S., Junejo, K. N., Mathur, A. A dataset to support research in the design of secure water treatment systems. Critical Information Infrastructures Security (CRITIS 2016), Springer, 2017.

[9] Kravchik, M., Shabtai, A. Detecting cyber attacks in industrial control systems using convolutional neural networks. Workshop on Cyber-Physical Systems Security and PrivaCy (CPS-SPC), ACM, 2018. arXiv:1806.08110

[10] Beyer, B., Jones, C., Petoff, J., Murphy, N. R. (eds.). Site Reliability Engineering: How Google Runs Production Systems. O'Reilly, 2016.

[11] Morris, K. Infrastructure as Code: Dynamic Systems for the Cloud Age, 2nd ed. O'Reilly, 2020.

[12] MITRE. ATT&CK for Industrial Control Systems. https://attack.mitre.org/matrices/ics/ (accessed 2026-10-24).

[13] Pedregosa, F., et al. Scikit-learn: Machine learning in Python. Journal of Machine Learning Research 12 (2011) 2825-2830.